Model targets and feature engineering (proposal)

Proposal for what the ML side predicts and which features to build from the cleaned tables in `s3://sagemaker-aep-outage-indiansassemble/processed/`. Builds on mvp-plan.md §5 and feature-specs.md. No code yet.

---

1. What the data told us (checked 2026-10-24, read from S3 in memory)

Finding	Number	What it changes
Outage labels are the same for every home on a transformer	Only 1 of ~27k transformers has mixed labels in Feb 2025	Train on **network nodes** (transformers / zones), not 117k premise rows that just repeat each other
Overhead vs underground **service drop** barely matters	Feb 2025 out: 19.4% overhead vs 18.3% underground	Outages come from **upstream** primary damage. Risk lives on the Power Path, not at the house
Storms hit different places	Zone-level rank correlation Feb 2025 vs Sep 2024 = **0.09** (vs "other" = 0.23)	"Damage history" will be a weak feature, and the Sep 2024 backtest will be humbling. Good material for *Navigating challenges*
Storm footprint is very uneven	Feb 2025 outage share per circuit: median 3%, max 97%	Raw lat/lon would just memorize the Feb footprint. **Don't use lat/lon; cross-validate grouped by circuit**
Damage is sparse	Feb 2025: 1,254 reports in 475 of 6,664 zones	Count models need to handle many zeros
Access issues are stable across storms	31.8% / 34.0% / 33.3% (Feb / Sep / other)	A base rate already works; the model has to beat it to earn its place

---

2. What to predict

#	Target	Unit	Label source	Model	Powers
A	**Probability a network link is damaged** in a storm	Each link on a chain: zone (fuse / recloser area) and transformer	`damage.outage_loc` / `xf_zone` / `transformer_loc` per event	Logistic regression (explainable). Stretch: HistGradientBoosting + SHAP	Per-home risk, **Your Power Path** (risk per link, weakest link)
A→	**Per-home outage risk** (derived, not	Premise	Validated against `outage_labels`	`P(out) = 1 − Π(1 − p_link)` over the links on the home's `chain`	Risk card, band, confidence, top reasons

#	Target	Unit	Label source	Model	Powers
	a separate model)				
B	**Expected damage count by type**	Zone × type (pole, primary wire, transformer, device, secondary)	`zone_damage_counts`	Poisson GLM with `log(overhead km)` as offset (damage per km of exposure). Negative binomial if overdispersed	Restoration simulator damage draws
C	**Probability a repair has an access problem**	Damage report	`access_labels`	Logistic regression; fall back to the 32% base rate if AUC < ~0.6	Simulator access adder
D	Restoration range	Premise	Not trained (simulator, as planned)	Monte Carlo over B and C + labor workbook	Range, "what moves it", **Time That Matters**
D'	Repair-lag check (optional)	Damage report	`repaired_date − report_date` (day level) by asset type	Empirical quantiles, no model	Sanity check that the simulator's spread looks like Feb 2025

Why A instead of a direct transformer model (MVP plan §5.1): the label is literally "something on this home's chain broke". Modelling each link and multiplying along the chain gives the home score *and* the Power Path link scores from one model, and the explanation becomes concrete: "the riskiest link is fuse F-881: 1.8 mi of overhead line, poles average 54 years". A direct transformer logistic regression stays as the baseline to beat.

Time That Matters needs no extra model: `P(outage > item limit)` is the share of simulator runs past each limit.

Storm severity: fit A and B pooled over all three events with an **event intercept**. The fitted intercepts give data-driven presets ("Sep-2024-like", "Feb-2025-like") instead of guessed 0.5×–2.0× multipliers. Keep the slider, but its stops come from data.

What the app needs back from each model (output contract)

Taken from the dashboard's mock API (`mock/api/*.json`, read by `web/app.js`), so the models plug in without UI changes.

Model	Output per row	Rows	Used by
A→ risk	`probability`, `band` (Low < 15% · Moderate 15–40% · High 40–70% · Very High > 70%), `confidence` + `confidence_reason`, `top_factors[]` = {`key`, `label`, `detail`, `direction`, `weight`}, `fallback`	Every premise × severity `0.5` / `1.0` / `1.5` / `2.0`	`risk.json`, `risk_by_severity.json`, `risk_fallback.json` (47 premises with no chain)
A per link	`p_damage` + one-line reason + rounded homes sharing it	Every transformer and zone on a chain	Power Path (not in the UI yet)
B damage	Expected count for `pole`, `primary_wire`, `transformer`, `device`, `secondary`	Every zone × severity	Restoration simulator → `restoration.json`
C access	`p_access` (the "Auto (38% chance)" in `compare_examples.json`)	Every zone	Restoration simulator

Note: the mock's top factor "fed by an overhead line" refers to the service drop, which the data says barely matters (§1). The real top factor should be upstream overhead line length.

---

3. Features (by source table)

All features are per **zone** unless marked. Transformer-level rows inherit their zone's features plus their own.

Exposure (`line`, batch 3)

- Overhead primary km in zone; underground share of primary
- Overhead secondary km in zone
- **Upstream exposure:** overhead primary km on every zone between this one and the breaker (walk `device.upstream_loc` / `chain`). Likely the strongest single feature
- Circuit total overhead km; voltage class

Structure condition (`pole`, batch 3)

- Pole count and poles per km of overhead line
- Mean age, share > 50 years, **share with known install year** (only ~50% known, so always paired with the flag)
- Wood share (`material_group`), mean height, class mix
- Note: 22.5% of poles have no zone (> 1 km from customer transformers) and are dropped

Protection and topology (`device`, `chain`, `zone`, batch 1)

- Zone's device type (fuse / recloser / breaker-only `BREAKER_CIRxx`), `automated`, `rating`
- Chain depth (protective devices between home and breaker), number of reclosers upstream
- `customers_downstream` of the zone device (log); transformers per zone

- Straight-line distance from zone to its breaker (proxy for "end of the line")

Equipment and customers (`transformer`, `premise`, batch 1)

- Transformer `kva`, `mounting` (pole vs pad), `phase`, customers per transformer (transformer-level rows)
- Customers in zone, customers per km of line (rural vs town), overhead service share, `city`

History (`damage`, batch 2), leave-one-event-out only

- Damage reports in *other* events in this zone and within 1 km
- Must exclude the event being predicted, or it leaks the label. Expect a small effect (see §1)

Access model C only (per damage report)

- `asset_type` / `damage_cat`, customers and poles within 500 m (density), zone underground share, upstream overhead km, `city` known
- **Excluded:** `special_equipment` (written on the same report, leaks), repair dates and `repaired`

Deliberately not used

- Raw lat/lon (memorizes the storm footprint); Life Support or priority data (privacy; app-only)

---

4. Validation (feeds *Prove it* and the confidence level)

- **Main test:** train on Feb 2025, test on Sep 2024 (and "other"). Report AUC, a calibration chart ("of homes we called High, X% lost power") and the Brier score, for model A→ vs the direct transformer baseline vs a naive "overhead km only" rule.
- **Within-storm CV:** 5 folds grouped by `circuit_id`, so the model can't learn a circuit's footprint.
- B: rank correlation of zones and error on total counts by type, on Sep 2024.
- C: AUC vs the 32% base rate. If it doesn't beat it, say so and use the base rate.
- Output per model: `model.joblib`, `metrics.json`, `features.json`, coefficient table (for plain-language reasons), as in MVP plan §10b.

---

5. Build order

1. **Shared feature table** (this step): one row per link per event for A, with a sibling zone×event frame for B and a damage-report frame for C derivable from the same joins. Written to `s3://.../processed/features/`.
2. Model A (link damage) + A→ derivation and baseline.
3. Model B (expected damage counts).
4. Model C (access problem).
5. Restoration simulator (D) wiring + Time That Matters probabilities.